Lab 1

Part A – Conceptual Understanding

Task A1: Waterfall Model Overview

1. Definition of the Waterfall Model
2. Explanation of each phase: • Requirements Analysis • System Design • Implementation • Integration & Testing • Deployment • Maintenance

1. Definition of the Waterfall Model

The Waterfall Model is a traditional, linear, and sequential software development life cycle (SDLC) model in which development progresses through a series of clearly defined phases. Each phase must be completed and approved before the next phase begins, and there is little to no overlap between phases.

It is called the *Waterfall Model* because progress flows downward step by step, similar to a waterfall. This model is best suited for projects where requirements are clearly defined, stable, and unlikely to change.

2. Phases of the Waterfall Model

1. Requirements Analysis

Description:
This is the first and most critical phase of the Waterfall Model. In this phase, the development team gathers and analyzes the functional and non-functional requirements from stakeholders, clients, and end users.

Inputs:

- Client requirements
- Business needs
- Stakeholder expectations

Activities:

- Requirement gathering through meetings, interviews, and surveys
- Feasibility analysis
- Requirement documentation
- Validation and approval of requirements

Outputs:

- Software Requirement Specification (SRS) document
- Approved and finalized requirements

2. System Design

Description:
In this phase, the system architecture and design are created based on the approved SRS. The focus is on how the system will be built, including hardware, software, database, and interface design.

Inputs:

- SRS document
- Technical constraints
- System standards

Activities:

- High-level system architecture design
- Low-level module design

- Database and interface design
- Selection of technologies and tools

Outputs:

- System Design Document (SDD)
- Architectural diagrams
- Database schemas

3. Implementation

Description:
During this phase, the actual coding and development of the system takes place. Each module is developed separately according to the design specifications.

Inputs:

- System Design Document
- Coding standards and guidelines

Activities:

- Writing source code
- Unit testing of individual modules
- Code reviews

Outputs:

- Source code
- Unit-tested software modules

4. Integration & Testing

Description:
In this phase, individual modules are integrated together and tested as a

complete system to identify defects and ensure the system meets the specified requirements.

Inputs:

- Developed software modules
- Test plans and test cases
- SRS document

Activities:

- Module integration
- System testing
- Functional and non-functional testing
- Bug identification and fixing

Outputs:

- Tested and validated system
- Test reports
- Defect logs

5. Deployment

Description:
Once the system is tested and approved, it is released to the production environment and made available to end users.

Inputs:

- Tested software system
- Deployment plan
- User manuals

Activities:

- Installation and configuration
- User training
- Data migration (if required)

Outputs:

- Deployed software
- Operational system
- User documentation

6. Maintenance

Description:
This is the final and longest phase of the Waterfall Model. It involves post-deployment support, including bug fixes, performance improvements, and system enhancements.

Inputs:

- User feedback
- Error reports
- Change requests

Activities:

- Bug fixing
- System updates and patches
- Performance optimization
- Minor enhancements

Outputs:

- Updated and improved software
- Maintenance reports

Conclusion

The Waterfall Model is simple and easy to manage due to its structured approach and clear documentation. However, it is not flexible to changes, making it less suitable for projects with evolving requirements. Despite its limitations, it is still widely used in projects with well-defined and stable requirements.

Part B – Case Study Application

Task B1: Case Study Selection

Simple E-Commerce Website

The system allows users to browse products, add them to a cart, place orders, and make payments online. Admin users can manage products, orders, and users.

Task B2: Apply Waterfall Phases

For the selected system, teams must map each Waterfall phase:

Phase	Team Activity
Requirements	List functional & non-functional requirements
Design	High-level architecture + module list
Implementation	Languages/tools assumed
Testing	Types of tests planned
Deployment	How system is released
Maintenance	Possible future changes

1. Requirements Phase

Functional Requirements (FR)

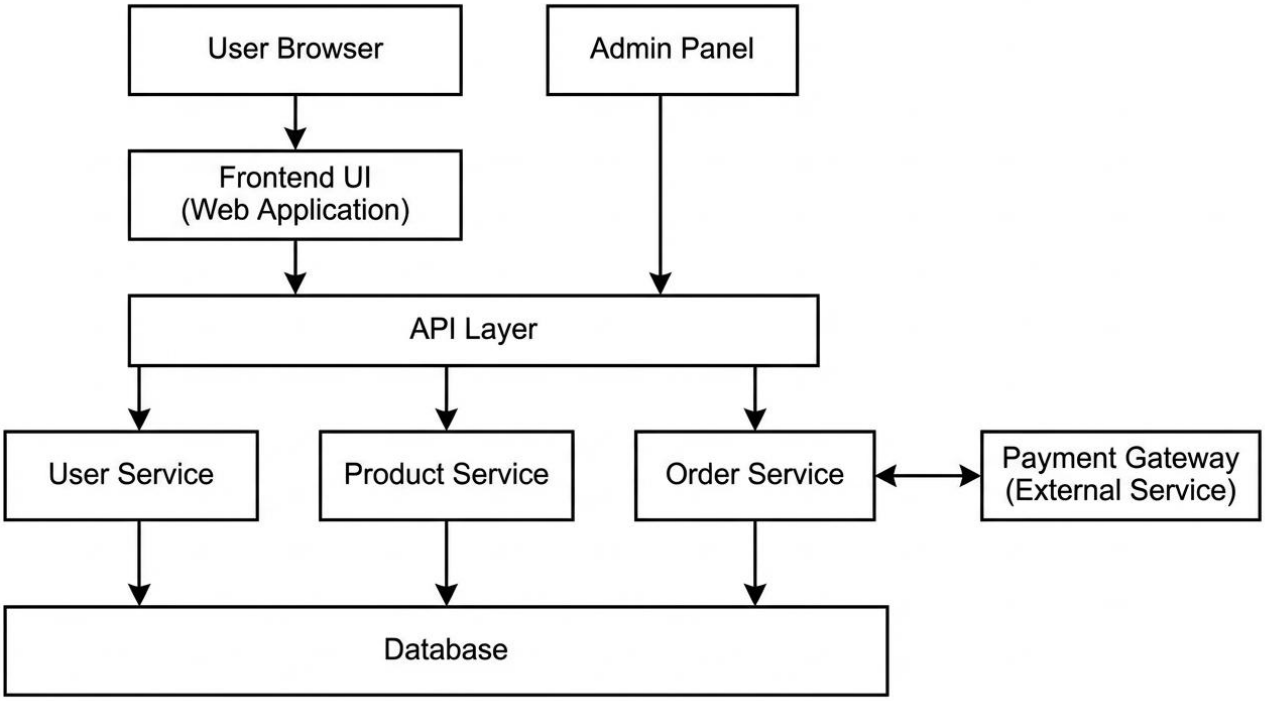
a) Users should be able to register and log in to the system
b) Users should be able to browse products by category
c) Users should be able to view product details (price, description, image)
d) Users should be able to add/remove products from the cart
e) Users should be able to place an order
f) Users should be able to make online payments
g) Users should be able to view order history
h) Admin should be able to add, update, and delete products
i) Admin should be able to view and manage customer orders

Non-Functional Requirements (NFR)

a) The system should be secure (authentication & authorization)
b) The system should have good performance (fast page load)
c) The system should be available 24/7
d) The system should be user-friendly and responsive

2. Design Phase

High-Level Architecture



Module List

1. User Authentication Module
2. Product Management Module
3. Shopping Cart Module
4. Order Management Module
5. Payment Processing Module
6. Admin Dashboard Module
7. Database Module

3. Implementation Phase

Assumed Languages & Tools

- Frontend: HTML, CSS, JavaScript
- Backend: Python (Django) / Java (Spring Boot)
- Database: MySQL
- Server: Apache / Tomcat
- Version Control: Git

Each module is developed separately based on the design documents.

4. Testing Phase

Types of Testing Planned

1. Unit Testing – Testing individual modules
2. Integration Testing – Testing interaction between modules
3. System Testing – Testing complete system functionality
4. Functional Testing – Verifying requirements from SRS
5. Security Testing – Testing login, payment, and data security
6. User Acceptance Testing (UAT) – Client validation

5. Deployment Phase

- The application is deployed on a cloud or hosting server
- Database is configured on the production server
- Domain name is connected
- Users access the website through a browser
- Admin credentials are provided separately

6. Maintenance Phase

Possible Future Changes

1. Adding mobile app support
2. Introducing discounts and offers
3. Adding multiple payment gateways
4. Improving UI/UX design
5. Performance optimization
6. Bug fixes and security patches

Assumptions

1. Internet connectivity is always available
2. Requirements are fixed before development starts
3. Payment gateway API is reliable
4. Users have basic knowledge of online shopping
5. Admin manages all backend operations

Conclusion

This case study demonstrates how a Simple E-Commerce Website can be developed using the Waterfall Model, where each phase is completed sequentially with proper documentation. The model works well here because the system requirements are clearly defined and stable.

Part C – Critical Analysis

Task C1: Advantages & Disadvantages

Each team must analyze Waterfall in the context of their case study, not generically.

Advantages of Using Waterfall for the Simple E-Commerce Website

1. Suitable for Stable and Well-Defined Requirements

In a simple e-commerce website, core features such as user registration, product listing, cart, orders, and payments are standard and well-understood. Since these requirements are unlikely to change during development, the Waterfall model works effectively by finalizing all requirements upfront before design and implementation.

Justification:
For this system, freezing requirements early reduces confusion and ensures smooth development without frequent redesigns.

2. Clear Structure and Easy Project Management

The Waterfall model divides development into clearly defined phases such as requirements, design, implementation, testing, and deployment. This makes it easier for the team to plan, assign responsibilities, and track progress.

Justification:
In the e-commerce system, different teams (frontend, backend, database) can work sequentially with clear deliverables for each phase, reducing coordination issues.

3. Strong Documentation Support

Each phase in the Waterfall model produces detailed documentation such as SRS, design documents, and test reports. This is highly beneficial for systems like e-commerce platforms that require future maintenance and upgrades.

Justification:
If new developers join later or changes are required (e.g., adding offers or new payment methods), the documentation helps understand the existing system easily.

Disadvantages of Using Waterfall for the Simple E-Commerce Website

1. Difficulty in Handling Requirement Changes

If new features such as wishlist, product reviews, or additional payment gateways are requested after development has started, accommodating these changes becomes difficult and expensive.

Justification:
For the e-commerce system, changing requirements would require revisiting earlier phases like design and implementation, increasing cost and time.

2. Late Testing Increases Risk

In the Waterfall model, testing is performed only after the complete implementation phase. This may cause critical issues to be discovered very late.

Justification:
For example, payment failures or cart calculation errors in the e-commerce website might be identified only during system testing, leading to major rework.

3. Limited Customer Feedback During Development

Customer involvement is minimal after the requirements phase. Users get to see the system only after deployment.

Justification:
If users find the UI confusing or the checkout process complex, these usability issues are identified too late, making improvements difficult without redesign.

Conclusion

While the Waterfall model is suitable for a simple e-commerce website with stable requirements and limited scope, it becomes less effective if frequent changes, continuous feedback, or rapid feature additions are required. For small and well-defined systems, Waterfall offers simplicity and control, but for evolving e-commerce platforms, Agile methods may be more appropriate.

Team Members

Siddeshwarprasad K R (1RUA24CSE0454)

Shreyas C P (1RUA24CSE0443)

Nikhil Savanth K M (1RUA24CSE0282)

Ullas Gowda R (1RUA25CSE7017)